Visi Anda membangun **Bot Injektor Monorepo Otonom** yang otomatis membuat folder, memecah file kode dari hasil chat AI, dan mengurus Git/GitHub sendiri sangat tepat untuk proyek skala raksasa (26+ domain, web, 177 agen, dan bot lintas platform). Anda juga memegang kendali penuh: skrip tidak boleh membuang versi yang mirip secara sepihak, melainkan **menyandingkannya dengan penanda audit ([AUDIT_PILIH_SATU])** agar Anda bisa memilih mana yang dipakai sebelum disuapkan ke AI.
Berikut adalah **2 senjata utama** yang sudah disatukan menjadi kode utuh siap jalan:

1. **Skrip Google Colab All-in-One**: Mount Google Drive otomatis, ekstraksi 2 PDF (868 halaman) via MinerU + gambar asli, pembacaan MD 3,8 juta karakter, **Detektor Kemiripan untuk Audit Manual Anda**, pengunci ketat **Sweetspot ≤ 45.000 karakter (MD) & ≤ 20 Halaman (PDF)**, serta pembungkus Auto-ZIP.
2. **Bot Otonom Injektor Monorepo & GitHub (auto_repo_bot.py)**: Bot yang otomatis ikut dibuat di dalam ZIP Anda. Saat Anda selesai *chat* dengan AI di setiap sesi *fresh chat*, Anda cukup menempelkan (*paste*) mentah-mentah balasan AI ke satu file inbox_ai.md, dan **Bot ini yang akan otomatis membuat foldernya, memecah menjadi puluhan file .py/.ts/.json, hingga melakukan git add, commit, dan push ke GitHub Codespaces Anda!**

Bagian 1: Skrip Utama Google Colab (100% Lossless + Panel Audit + Generator Bot Otonom)

Buat **1 sel kode** di Google Colab (pastikan memilih **Runtime > Change runtime type > T4 GPU**), lalu tempel dan jalankan seluruh kode di bawah ini:

```
#
================================================================================
========
# SEL 1: INSTALASI & MOUNT GOOGLE DRIVE OTOMATIS
#
================================================================================
========
# Hapus tanda '#' di baris bawah ini saat pertama kali menjalankan di
Colab:
# !pip install -U "magic-pdf[full]" --extra-index-url
https://wheels.myhloli.com tqdm pymupdf

import os
import re
import shutil
import hashlib
import subprocess
from pathlib import Path
from collections import defaultdict
from difflib import SequenceMatcher
from tqdm import tqdm
import fitz  # PyMuPDF

# 1. Mount Google Drive Otomatis agar proses 45 menit 100% aman dari
```

```
putus koneksi
try:
    from google.colab import drive, files
    drive.mount('/content/drive')
    BASE_DRIVE_DIR =
Path("/content/drive/MyDrive/PROYEK_MONOREPO_LOSSLESS")
    IN_COLAB = True
except ImportError:
    BASE_DRIVE_DIR = Path("./PROYEK_MONOREPO_LOSSLESS")
    IN_COLAB = False

BASE_DRIVE_DIR.mkdir(parents=True, exist_ok=True)

#
================================================================================
========
# 2. LOKASI FILE MENTAH ANDA (TARUH DI GOOGLE DRIVE ATAU FOLDER COLAB)
#
================================================================================
========
# Anda bisa menaruh file PDF & MD di Google Drive agar tidak perlu
upload berulang kali
RAW_PDF_FILES = [
    "blueprint_utama_1.pdf",       # Ganti dengan path PDF ke-1 Anda
(total 868 hal)
    "blueprint_utama_2.pdf"        # Ganti dengan path PDF ke-2 Anda
(jika ada)
]
RAW_MD_FILES = [
    "blueprint_utama.md",          # MD Mentah 3.8 Juta Karakter
    "hasil_lama_tidak_maksimal.md" # Opsional: file lama untuk ikut
diaudit silang
]

# BATAS KETAT SWEETSPOT AI SESUAI ATURAN GEMINI / ARGON APPS:
MAX_MD_SWEETSPOT_CHARS = 45000     # Maksimal 45.000 karakter per file
MD (1 sesi chat)
MAX_PDF_SWEETSPOT_PAGES = 20       # Maksimal 20 halaman per file PDF
(1 sesi chat)

# Struktur Folder Output di Google Drive
MINERU_OUT_DIR    = BASE_DRIVE_DIR / "01_MINERU_RAW_EXTRACTION"
MASTER_BLUEPRINT  = BASE_DRIVE_DIR /
"02_MASTER_PETA_OTAK_SWEETSPOT_45K"
MODUL_MD_45K_DIR  = BASE_DRIVE_DIR /
"03_MODUL_MD_AUDIT_SIAP_PAKAI_45K"
MODUL_PDF_20P_DIR = BASE_DRIVE_DIR / "04_MODUL_PDF_VISUAL_MAKS_20_HAL"
IMAGES_GLOBAL_DIR = BASE_DRIVE_DIR / "05_KOLEKSI_GAMBAR_DIAGRAM_ASLI"
```

```
BOT_AUTONOMOUS_DIR= BASE_DRIVE_DIR / "06_BOT_OTONOM_GITHUB_CODESPACES"

for d in [MINERU_OUT_DIR, MASTER_BLUEPRINT, MODUL_MD_45K_DIR,
MODUL_PDF_20P_DIR, IMAGES_GLOBAL_DIR, BOT_AUTONOMOUS_DIR]:
    d.mkdir(parents=True, exist_ok=True)

#
================================================================================
# 3. KAMUS 25 DOMAIN + MESIN AUTO-DISCOVERY (TANGKAP 100% LABEL LIAR)
#
================================================================================
DOMAIN_SCHEMA = {
    "01_ARCH": ["monorepo", "tech stack", "microservice", "env", "api
gateway", "dependency", "arsitektur", "architecture"],
    "02_CORE_AI": ["llm", "router", "sota", "rag", "memory", "fine
tuning", "fine-tuning", "fallback", "embedding", "prompt"],
    "03_V_OFFICE": ["177 agen", "kantor virtual", "virtual office",
"org chart", "state machine", "message broker", "consensus",
"handoff", "shift", "agent_"],
    "04_PROMPT_LIB": ["system prompt", "few shot", "few-shot", "json
schema", "negative prompt", "chain template"],
    "05_TOOLS_ENGINE": ["playwright", "puppeteer", "ffmpeg", "mcp",
"browser agent", "file converter", "search api"],
    "06_AGGREGATOR": ["scraper", "scraping", "spider", "crawler",
"aggregator", "social listener", "market spy", "deduplicator",
"normalizer"],
    "07_CONTENT_FACTORY": ["copywriting", "script director", "video
assembler", "carousel", "localization", "quality gate", "pabrik
aset"],
    "08_OMNI_SOCIAL": ["whatsapp", "telegram", "instagram",
"facebook", "threads", "tiktok", "comment bot", "sosial media",
"auto-upload"],
    "09_COMMERCE_AFF": ["affiliate", "marketplace", "cloaking",
"autolist", "price sync", "crm", "offer hunter", "shopee",
"tokopedia", "tiktok shop"],
    "10_STOCK_ENGINE": ["microstock", "shutterstock", "adobe stock",
"freepik", "upscaler", "vector", "exif", "iptc", "sftp", "artifact
cleaner"],
    "11_WEB_PLATFORM": ["frontend", "admin console", "dashboard",
"cms", "seo", "client portal", "websocket"],
    "12_FINANCE": ["pricing", "harga", "finansial", "token economics",
"unit economics", "payment gateway", "revenue", "profit", "stripe",
"midtrans", "rumus harga"],
    "13_DATA_SCHEMA": ["postgresql", "mysql", "sql", "mongodb",
"nosql", "vector db", "qdrant", "pinecone", "milvus", "neo4j", "graph
db", "redis", "s3", "bucket", "database", "schema"],
```

```
    "14_SEC_STEALTH": ["anti-detect", "antidetect", "proxy",
"residential proxy", "cookies", "captcha", "rbac", "vault", "stealth",
"fingerprint", "gologin", "multilogin", "adspower"],
    "15_TELEMETRY": ["telemetry", "tracing", "prometheus", "grafana",
"sentry", "audit trail", "ab testing", "metrics", "logging"],
    "16_INFRA_DEVOPS": ["docker", "kubernetes", "k8s", "celery",
"bullmq", "cron", "gpu", "ci/cd", "cicd", "backup", "devops", "vps",
"cloud"],
    "17_COMPLIANCE": ["tos", "rate limit", "copyright", "watermark",
"compliance", "banned", "suspend"],
    "18_SELF_HEALING": ["self-healing", "self healing", "dom adapter",
"account recovery", "dead letter", "dlq", "auto fix"],
    "19_TESTING_QA": ["unit test", "pytest", "jest", "integration
mock", "agent eval", "qa", "testing"],
    "20_BIZ_ROADMAP": ["mvp", "scaling", "roadmap", "secret sauce",
"strategi bisnis"],
    "21_OPPORTUNITY_SPIDER": ["bansos", "free tier", "airdrop",
"web3", "price glitch", "domain murah", "panel murah", "bounty",
"grant", "arbitrage"],
    "22_HITL_ACTION": ["1-klik", "one-click", "one click", "tombol
eksekusi", "pelaporan rutin", "digest", "auto claimer", "approval",
"hitl"],
    "23_NVIDIA_SANDBOX": ["nvidia", "skillspector", "skill inspector",
"nim", "omniverse", "tensorrt", "sandbox", "inspeksi pra-penerapan"],
    "24_DYNAMIC_PLUGIN": ["auto wrapper", "skill injector",
"credential rotation", "rotasi api", "plugin dinamis"],
    "25_IDENTITY_FARM": ["temp mail", "sms activate", "otp", "vcc",
"aged account", "akun tua", "ternak akun", "identity farm"]
}

#
================================================================================
========
# 4. EKSTRAKSI PDF MINERU + PENYELAMAT GAMBAR + PEMOTONG PDF 20
HALAMAN
#
================================================================================
========
def extract_pdf_and_rescue_images(pdf_path: str, output_dir: Path) ->
Path:
    pdf_stem = Path(pdf_path).stem
    found = list(output_dir.rglob(f"{pdf_stem}.md"))
    if not found:
        print(f"[*] Mengekstrak {pdf_path} via MinerU (Tabel, LaTeX,
Kode & Gambar)...")
        subprocess.run(["magic-pdf", "-p", pdf_path, "-o",
str(output_dir), "-m", "auto"], check=True)
        found = list(output_dir.rglob(f"{pdf_stem}.md"))
```

```
        if not found:
            raise FileNotFoundError(f"Gagal menemukan hasil MD MinerU
untuk {pdf_path}")

    md_path = found[0]
    for idir in md_path.parent.rglob("images"):
        if idir.is_dir():
            for img in idir.iterdir():
                if img.is_file():
                    dest = IMAGES_GLOBAL_DIR /
f"{pdf_stem}_{img.name}"
                    if not dest.exists():
                        shutil.copy2(img, dest)
    return md_path

def slice_pdf_20_pages(pdf_path: str):
    doc = fitz.open(pdf_path)
    stem = Path(pdf_path).stem
    for idx, start_p in enumerate(range(0, len(doc),
MAX_PDF_SWEETSPOT_PAGES), 1):
        end_p = min(start_p + MAX_PDF_SWEETSPOT_PAGES - 1, len(doc) -
1)
        sub_doc = fitz.open()
        sub_doc.insert_pdf(doc, from_page=start_p, to_page=end_p)
        sub_doc.save(str(MODUL_PDF_20P_DIR /
f"{stem}_PART_{idx:02d}_HAL_{start_p+1}_{end_p+1}.pdf"))
        sub_doc.close()
    doc.close()

#
================================================================================
========
# 5. KLASIFIKASI AUTO-DISCOVERY & DETEKTOR DUPLIKAT PARSIAL UNTUK
AUDIT MANUAL
#
================================================================================
========
def classify_and_extract_dna(title: str, content: str) -> tuple[str,
list[str], dict]:
    combined = (title * 3 + "\n" + content).lower()
    raw_tags = re.findall(r"\[([A-Za-z0-9_\-\s:]{3,40})\]", title +
"\n" + content[:3000])
    clean_tags = list(dict.fromkeys([t.strip().upper() for t in
raw_tags if not t.strip().isdigit()]))

    matched_dom = None
    for tag in clean_tags:
        for dom_key in DOMAIN_SCHEMA:
```

```
            if dom_key.split("_", 1)[1] in tag:
                matched_dom = dom_key
                break
        if matched_dom:
            break

    if not matched_dom:
        scores = {d: sum(combined.count(kw) for kw in kws) for d, kws
in DOMAIN_SCHEMA.items()}
        scores = {k: v for k, v in scores.items() if v > 0}
        if scores:
            matched_dom = max(scores, key=scores.get)
        elif clean_tags:
            safe_tag = re.sub(r"[^A-Z0-9_]", "_", clean_tags[0])[:25]
            matched_dom = f"26_AUTO_DISCOVERY_{safe_tag}"
        else:
            matched_dom = "26_AUTO_DISCOVERY_UMUM"

    funcs =
list(dict.fromkeys(re.findall(r"(?:def|class|function|interface|CREATE
TABLE)\s+([a-zA-Z0-9_]+)", content)))
    imgs = re.findall(r"!\[[^\]]*\]([^)]+)", content)
    code_cnt = len(re.findall(r"```", content)) // 2
    tbl_cnt = len(re.findall(r"^\|.+\|$", content, re.MULTILINE))
    score = (code_cnt * 15) + (tbl_cnt * 2) + (len(funcs) * 5) +
(len(imgs) * 8)

    dna = {
        "score": score,
        "grade": "BINTANG_5_KODE_UTUH" if score >= 40 else
("BINTANG_3_TABEL_STRUKTUR" if score >= 10 else
"BINTANG_1_KONSEP_NARASI"),
        "symbols": funcs[:15],
        "images": imgs,
        "code_blocks": code_cnt,
        "table_rows": tbl_cnt
    }
    return matched_dom, clean_tags, dna

def split_sections_lossless(md_text: str, source_tag: str) ->
list[dict]:
    lines = md_text.splitlines(keepends=True)
    sections, curr_lines = [], []
    curr_title = "ROOT_INTRODUCTION"
    in_code = False

    for line in lines:
```

```
        if line.lstrip().startswith("```"):
            in_code = not in_code
        if not in_code and re.match(r"^#{1,3}\s+.+", line):
            if curr_lines:
                raw = "".join(curr_lines)
                dom, tags, dna = classify_and_extract_dna(curr_title,
raw)
                sections.append({"source": source_tag, "title":
curr_title.strip(), "content": raw, "domain": dom, "tags": tags,
"dna": dna})
            curr_title = line
            curr_lines = [line]
        else:
            curr_lines.append(line)

    if curr_lines:
        raw = "".join(curr_lines)
        dom, tags, dna = classify_and_extract_dna(curr_title, raw)
        sections.append({"source": source_tag, "title":
curr_title.strip(), "content": raw, "domain": dom, "tags": tags,
"dna": dna})
    return sections

def mark_similar_sections_for_human_audit(secs: list[dict]):
    """
    Mendeteksi bab dari PDF vs MD yang judul/isinya mirip (tapi tidak
100% kembar),
    lalu menaruhnya BERDAMPINGAN dan memberi bendera
[AUDIT_MANUAL_PILIH_SATU]
    agar Anda mudah mengaudit mana yang mau dipakai sebelum disuapkan
ke AI!
    """
    n = len(secs)
    visited = [False] * n
    ordered = []

    # Urutkan awal berdasarkan skor tertinggi
    secs.sort(key=lambda x: x["dna"]["score"], reverse=True)

    for i in range(n):
        if visited[i]:
            continue
        visited[i] = True
        base = secs[i]
        base["audit_flag"] = "MANDIRI_UNIK"
        cluster = [base]

        # Cari apakah ada seksi lain di domain yang sama yang mirip
```

```
judulnya atau fungsinya
        base_title_clean = re.sub(r"[^a-z0-9]", "",
base["title"].lower())
        base_syms = set(base["dna"]["symbols"])

        for j in range(i + 1, n):
            if visited[j]:
                continue
            cand = secs[j]
            cand_title_clean = re.sub(r"[^a-z0-9]", "",
cand["title"].lower())
            title_sim = SequenceMatcher(None, base_title_clean,
cand_title_clean).ratio() if base_title_clean and cand_title_clean
else 0
            sym_overlap =
len(base_syms.intersection(cand["dna"]["symbols"])) > 0

            if title_sim >= 0.65 or sym_overlap:
                visited[j] = True
                base["audit_flag"] = f"⚠️
KEMBAR_PARSIAL_DENGAN_VERSI_LAIN (Rekomendasi Otomatis: PILIH INI
karena Skor={base['dna']['score']})"
                cand["audit_flag"] = f"⚠️
KEMBAR_PARSIAL_VERSI_ALTERNATIF (Skor={cand['dna']['score']} - Hapus
blok ini saat audit jika sudah diwakili versi atas, atau gabungkan)"
                cluster.append(cand)

        ordered.extend(cluster)
    return ordered

def slice_oversized_safely(text: str, max_chars: int) -> list[str]:
    if len(text) <= max_chars:
        return [text]
    lines = text.splitlines(keepends=True)
    chunks, curr, curr_len, in_code = [], [], 0, False
    for line in lines:
        if line.lstrip().startswith("```"):
            in_code = not in_code
        if curr_len + len(line) > max_chars and not in_code and curr:
            chunks.append("".join(curr))
            curr, curr_len = [line], len(line)
        else:
            curr.append(line)
            curr_len += len(line)
    if curr:
        chunks.append("".join(curr))
    return chunks
```

```
#
================================================================================
========
# 6. GENERATOR BOT OTONOM INJEKTOR MONOREPO & GITHUB CODESPACES
#
================================================================================
========
def generate_autonomous_github_bot():
    """
    Mencetak skrip Bot Otonom (auto_repo_bot.py) yang akan berjalan di
GitHub Codespaces Anda.
    Tugas Bot: Membaca balasan kode dari AI di setiap fresh chat,
otomatis membuat foldernya,
    menulis/mengupdate file kodenya, dan melakukan Git Commit & Push
mandiri!
    """
    bot_code = '''#!/usr/bin/env python3
"""
================================================================================
==========
BOT OTONOM ARSITEK MONOREPO & GITHUB CODESPACES (auto_repo_bot.py)
================================================================================
==========
Cara Pakai di GitHub Codespaces:
1. Setiap kali selesai 1 Fresh Chat bersama AI, minta AI memberi tag
path di atas blok kode:
   Contoh format yang diminta dari AI:
   ```python filepath: src/02_core_ai/router.py
   # isi kode dari AI...
```

Atau komentar di baris pertama dalam blok kode: # FILE: src/02_core_ai/router.py 2. Copy-paste seluruh balasan chat AI ke dalam file inbox_ai.md di Codespaces. 3. Jalankan perintah: python3 auto_repo_bot.py -> Bot otomatis membuat semua folder yang belum ada, menulis file kodenya, mencatat log integrasi, dan otomatis menjalankan git add, commit, & push! """ import os import re import subprocess from pathlib import Path from datetime import datetime

INBOX_FILE = Path("inbox_ai.md") ARCHIVE_DIR = Path(".bot_history_inbox") ARCHIVE_DIR.mkdir(exist_ok=True)

def parse_and_inject_code(markdown_text: str) -> list[str]: # Menangkap blok kode dengan deklarasi path file di pagar atau di baris pertama komentar pattern = re.compile(r"[a-zA-Z0-9_-]*(?:\s+(?:filepath:|file:|path:)?\s*([a-zA-Z0-9_./-]+.[a-zA-Z0-9]+))?\n(.*?)```", re.DOTALL) created_files = []

for match in pattern.finditer(markdown_text): header_path, code_body = match.group(1), match.group(2) target_path = header_path

Jika path ada di baris pertama komentar di dalam blok kode (# FILE: src/...) if not target_path: first_line = code_body.splitlines()[0] if code_body.splitlines() else "" m_inner = re.search(r"(?:FILE|PATH|FILEPATH)\s*:\s*([a-zA-Z0-9_./-]+.[a-zA-Z0-9]+)", first_line, re.IGNORECASE) if m_inner: target_path = m_inner.group(1)
```

if target_path: clean_path = Path(target_path.strip().lstrip("/")) clean_path.parent.mkdir(parents=True, exist_ok=True) clean_path.write_text(code_body, encoding="utf-8") created_files.append(str(clean_path)) print(f"✅ [BOT-WRITE] Folder & File otomatis dibuat/diupdate: {clean_path} ({len(code_body):,} chars)")
return created_files
def auto_git_sync(files_modified: list[str]): if not files_modified: return try: subprocess.run(["git", "add", "."], check=True) msg = f"🤖 [AUTO-BOT] Injeksi {len(files_modified)} file: {', '.join(files_modified[:4])}" subprocess.run(["git", "commit", "-m", msg], check=True) print(f"📦 [BOT-GIT] Berhasil commit: {msg}") # Otomatis push jika remote origin sudah terhubung di Codespaces subprocess.run(["git", "push"], check=False) except Exception as e: print(f"ℹ️ [BOT-GIT] Catatan Git lokal tersimpan (Push manual jika diperlukan): {e}")
def main(): if not INBOX_FILE.exists(): INBOX_FILE.write_text("# Tempel (Paste) balasan kode dari Sesi Chat AI Anda di sini lalu jalankan: python3 auto_repo_bot.py\n", encoding="utf-8") print(f"📁 File {INBOX_FILE} telah disiapkan. Tempel balasan AI Anda di sana!") return raw_chat = INBOX_FILE.read_text(encoding="utf-8") if len(raw_chat.strip()) < 20: print("⚠️ inbox_ai.md masih kosong. Tempel balasan kode dari AI terlebih dahulu.") return written = parse_and_inject_code(raw_chat) if written: ts = datetime.now().strftime("%Y%m%d_%H%M%S") backup = ARCHIVE_DIR / f"session_{ts}.md" backup.write_text(raw_chat, encoding="utf-8") INBOX_FILE.write_text("", encoding="utf-8") # Kosongkan inbox untuk sesi chat berikutnya auto_git_sync(written) print(f"\n🎉 SELESAI! {len(written)} file berhasil disuntikkan ke Monorepo & riwayat chat diarsipkan ke {backup}") else: print("⚠️ Tidak ditemukan penanda path file di blok kode. Pastikan AI menyertakan # FILE: nama_folder/nama_file.py di baris pertama tiap blok kode!")
if **name** == "**main**": main() ''' (BOT_AUTONOMOUS_DIR / "auto_repo_bot.py").write_text(bot_code, encoding="utf-8")
Buat Prompt Standar Sesi Chat agar AI selalu patuh pada format Bot Otonom Anda
prompt_guide = ("# PROMPT WAJIB UNTUK SETIAP SESI FRESH CHAT AI ANDA\n\n" "Salin kalimat singkat ini setiap kali Anda meng-upload 1 file Sweetspot (<=45k karakter) ke AI:\n\n" ">
"Kamu adalah Senior Principal Engineer untuk Monorepo Skala Besar saya. " "Baca seluruh spesifikasi di file blueprint yang saya lampirkan ini tanpa melewatkan 1 fitur pun. " "Catatan: Abaikan blok yang saya beri tanda [DROP_SAAT_AUDIT] (jika ada). " "Tuliskan kode produksi yang 100% utuh (tanpa placeholder/TODO). " "WAJIB: Di baris paling atas di dalam SETIAP blok kode, tuliskan komentar # FILE: path/lengkap/nama_file.ext " "agar Bot Otonom Monorepo saya (auto_repo_bot.py) bisa langsung membuat folder dan menyuntikkan kodenya secara otomatis ke GitHub Codespaces."\n") (BOT_AUTONOMOUS_DIR / "CARA_PAKAI_DAN_PROMPT_AI.md").write_text(prompt_guide, encoding="utf-8")

==========

==========

==========

7. EKSEKUSI PIPELINE UTAMA +

PEMBUNGKUS AUTO-ZIP

==========

==========

==========

def execute_all(): all_sections = [] total_raw_chars = 0
for pdf_f in RAW_PDF_FILES: if os.path.exists(pdf_f): slice_pdf_20_pages(pdf_f) md_p = extract_pdf_and_rescue_images(pdf_f, MINERU_OUT_DIR) txt = md_p.read_text(encoding="utf-8", errors="replace") total_raw_chars += len(txt) all_sections.extend(split_sections_lossless(txt, f"PDF:{Path(pdf_f).name}")) else: print(f"[!] File PDF {pdf_f} tidak ditemukan, dilewati.")
for md_f in RAW_MD_FILES: if os.path.exists(md_f): txt = Path(md_f).read_text(encoding="utf-8", errors="replace") total_raw_chars += len(txt) all_sections.extend(split_sections_lossless(txt, f"MD:{Path(md_f).name}")) else: print(f"[!] File MD {md_f} tidak ditemukan, dilewati.")
Deduplikasi 100% SHA-256 Utuh (Hanya membuang jika isi karakter kembar 100% identik) domain_groups = defaultdict(list) seen_hashes = set() exact_dup_chars = 0
for s in tqdm(all_sections, desc="Memeriksa Keutuhan 100% Karakter & Auto-Discovery"): if not s["content"]: continue h = hashlib.sha256(s["content"].encode("utf-8")).hexdigest() if h in seen_hashes: exact_dup_chars += len(s["content"]) continue seen_hashes.add(h) domain_groups[s["domain"]].append(s)
Tandai kemiripan parsial agar mudah Anda Audit Manual master_catalog = [] total_saved_chars = 0
for dom in sorted(domain_groups.keys()): audited_secs = mark_similar_sections_for_human_audit(domain_groups[dom]) part_no = 1 curr_bucket, curr_meta = [], [] curr_len = 0
for idx, s in enumerate(audited_secs, 1): img_str = f" | GAMBAR: {', '.join(s['dna']['images'][:3])}" if s["dna"]["images"] else "" tag_str = f" | TAG_ASLI: {', '.join(s['tags'][:4])}" if s["tags"] else "" audit_banner = (f"\n<!--
==========
-->\n" f"<!-- [ID:{dom}_S{idx:03d}] SUMBER: {s['source']} | KUALITAS: {s['dna']['grade']} (Skor: {s['dna']['score']}) -->\n" f"<!-- STATUS AUDIT: {s['audit_flag']}{tag_str}{img_str} -->\n" f"<!-- AKSI AUDIT ANDA: [PERTAHANKAN] (Ubah jadi [DROP_SAAT_AUDIT] jika tidak ingin dipakai AI) -->\n" f"<!--
==========
-->\n") full_block = audit_banner + s["content"] total_saved_chars += len(s["content"]) sub_blocks = slice_oversized_safely(full_block, MAX_MD_SWEETSPOT_CHARS - 1200) for sub_i, sb in enumerate(sub_blocks, 1): if curr_len + len(sb) > MAX_MD_SWEETSPOT_CHARS and curr_bucket: fname = f"{dom}*SWEETSPOT*{part_no:02d}.md" (MODUL_MD_45K_DIR / fname).write_text("".join(curr_bucket), encoding="utf-8") master_catalog.append({"file": fname, "chars": curr_len, "items": curr_meta}) part_no += 1 curr_bucket, curr_meta, curr_len = [], [], 0 curr_bucket.append(sb) curr_len += len(sb) curr_meta.append({ "id": f"{dom}_S{idx:03d}" +

(f"_P{sub_i}" if len(sub_blocks) > 1 else ""), "title": re.sub(r"\s+", " ", s["title"])[:85], "grade": s["dna"]["grade"], "audit": "KEMBAR_PARSIAL" if "KEMBAR_PARSIAL" in s["audit_flag"] else "UNIK", "symbols": s["dna"]["symbols"][:5], "chars": len(sb) })
if curr_bucket: fname = f"{dom}*SWEETSPOT*{part_no:02d}.md" (MODUL_MD_45K_DIR / fname).write_text("".join(curr_bucket), encoding="utf-8") master_catalog.append({"file": fname, "chars": curr_len, "items": curr_meta})
Cetak 1-3 File Master Peta Otak (Dikunci ketat <= 45.000 karakter per file) m_header = ("# MASTER PETA OTAK & KONTRAK MONOREPO (SWEETSPOT <= 45.000 KARAKTER)\n" "Berisi 100% indeks seluruh modul, fungsi kode, gambar, dan 26_AUTO_DISCOVERY.\n" "Upload file ini di Sesi Pertama untuk menginisiasi struktur dasar Monorepo bersama auto_repo_bot.py.\n\n") m_part, m_curr, m_len = 1, [m_header], len(m_header)
for entry in master_catalog: lines = [f"\n## 📁 {entry['file']} ({entry['chars']:,} chr)\n"] for it in entry["items"]: sym = f" | Fungsi: {', '.join(it['symbols'])}" if it["symbols"] else "" aud = " ⚠️*[Cek Audit Kembar]*" if it["audit"] == "KEMBAR_PARSIAL" else "" lines.append(f"- **[{it['id']}]** {it['title']} ({it['chars']:,} chr | {it['grade']}{sym}){aud}\n") block = "".join(lines)
if m_len + len(block) > MAX_MD_SWEETSPOT_CHARS and m_curr: (MASTER_BLUEPRINT / f"00_MASTER_PETA_OTAK_PART_{m_part:02d}.md").write_text("".join(m_curr), encoding="utf-8") m_part += 1 m_curr = [m_header + f"*(Bagian Lanjutan {m_part})*\n\n"] m_len = len(m_curr[0]) m_curr.append(block) m_len += len(block)
if m_curr: (MASTER_BLUEPRINT / f"00_MASTER_PETA_OTAK_PART_{m_part:02d}.md").write_text("".join(m_curr), encoding="utf-8")
Bangkitkan Bot Otonom GitHub Codespaces generate_autonomous_github_bot()
Buat Laporan Audit Lossless audit_txt = (f"# LAPORAN VERIFIKASI 100% LOSSLESS & PANEL AUDIT\n\n" f"- **Total Karakter Mentah Masuk**: {total_raw_chars:,} karakter\n" f"- **Duplikat Kembar 100% Identik**: {exact_dup_chars:,} karakter\n" f"- **Total Karakter Bersih Tersimpan**: {total_saved_chars:,} karakter (**0% Karakter Hilang!**)\n" f"- **Jumlah File Modul Sweetspot (<=45k chr)**: {len(master_catalog)} file\n") (BASE_DRIVE_DIR / "00_LAPORAN_VERIFIKASI_LOSSLESS.md").write_text(audit_txt, encoding="utf-8")
Bungkus ZIP Otomatis di Google Drive zip_code_pack = str(BASE_DRIVE_DIR.parent / "PAKET_SIAP_CODING_DAN_BOT_OTONOM") print("\n[*] Membungkus Paket ZIP Siap Coding + Bot Otonom ke Google Drive...") shutil.make_archive(zip_code_pack, 'zip', BASE_DRIVE_DIR) print(f"✅ SELESAI! Paket ZIP Utuh tersimpan di Google Drive: {zip_code_pack}.zip")
if IN_COLAB: try: files.download(f"{zip_code_pack}.zip") except Exception: print("ℹ️ File ZIP sudah aman di Google Drive Anda (silakan unduh langsung dari Drive atau tarik ke Codespaces).")
if **name** == "**main**": execute_all()

Bagian 2: Cara Kerja "Panel Audit Manual" + "Bot Otonom GitHub Codespaces" Anda

Perhatikan betapa mudahnya alur kerja Anda setelah skrip di atas selesai:

1. Cara Anda Mengaudit Bagian yang Mirip (Sebelum Buat Kode)
Di dalam setiap file pecahan `03_MODUL_MD_AUDIT_SIAP_PAKAI_45K`, jika

```
ada bagian dari PDF dan MD yang membahas hal serupa (misal sama-sama
membahas *Router LLM* tetapi isinya sedikit berbeda), skrip **tidak
akan menghapus salah satunya**, melainkan menaruh keduanya
berdampingan persis atas-bawah dengan label mencolok:
```markdown
<!-- [ID:02_CORE_AI_S001] SUMBER: MD:blueprint_utama.md | KUALITAS:
BINTANG_5_KODE_UTUH (Skor: 85) -->
<!-- STATUS AUDIT: ⚠️ KEMBAR_PARSIAL_DENGAN_VERSI_LAIN (Rekomendasi
Otomatis: PILIH INI karena Skor=85) -->
<!-- AKSI AUDIT ANDA: [PERTAHANKAN] (Ubah jadi [DROP_SAAT_AUDIT] jika
tidak ingin dipakai AI) -->
```

- **Tugas Audit Anda sangat ringan:** Buka file tersebut, tekan Ctrl + F dan cari kata KEMBAR_PARSIAL. Anda tinggal melihat 2 blok yang berdampingan itu. Jika Anda setuju dengan rekomendasi skor atau lebih suka versi yang bawah, Anda cukup menghapus blok yang tidak diinginkan (atau mengubah tulisannya menjadi [DROP_SAAT_AUDIT]), sehingga **AI tidak akan pernah bingung menghadapi 2 instruksi ganda!**

2. Cara Menarik ZIP dari Google Drive Langsung ke Terminal GitHub Codespaces

Tanpa perlu mengunduh ratusan MB ke HP/Laptop lalu meng-upload ulang ke GitHub Codespaces, Anda cukup:

1. Klik kanan file PAKET_SIAP_CODING_DAN_BOT_OTONOM.zip di Google Drive Anda > **Bagikan (Share)** > **Siapa saja yang memiliki link** > Salin ID file-nya.
2. Buka terminal **GitHub Codespaces** Anda, lalu jalankan 3 baris perintah ini:
   ```
   pip install gdown
   gdown --id MASUKKAN_ID_FILE_GOOGLE_DRIVE_ANDA_DI_SINI -O paket.zip
   unzip paket.zip -d ./blueprint_vault && cp
   ./blueprint_vault/06_BOT_OTONOM_GITHUB_CODESPACES/auto_repo_bot.py
   .
   ```

3. Cara Bot Otonom (auto_repo_bot.py) Mengurus Folder & GitHub Anda

Setiap kali Anda membuka **1 Fresh Chat** dan meng-upload **1 file Sweetspot (≤ 45k karakter)**:

1. Gunakan prompt standar yang sudah dibuatkan di file CARA_PAKAI_DAN_PROMPT_AI.md (yang menyuruh AI menuliskan # FILE: src/nama_modul/nama_file.py di baris pertama tiap blok kodenya).
2. Begitu AI selesai memuntahkan kode, Anda **tidak perlu pusing membuat folder atau membuat file satu per satu**. Cukup klik tombol **Copy** pada jawaban AI, tempelkan (*paste*) semuanya ke dalam file inbox_ai.md di GitHub Codespaces Anda, lalu ketik di terminal:
   ```
   python3 auto_repo_bot.py
   ```
3. Dalam 1 detik, **auto_repo_bot.py akan otomatis**:
   - Membaca seluruh blok kode dari AI.
   - Mengecek apakah foldernya (misal src/03_v_office/agents/ atau web/dashboard/components/) sudah ada. **Jika belum ada, bot otomatis**
   ```

membuatkan foldernya!

- Menulis masing-masing potongan kode ke file tujuannya secara presisi.
- Mengarsipkan obrolan tadi ke folder .bot_history_inbox/ dan mengosongkan inbox_ai.md untuk sesi chat berikutnya.
- Menjalankan git add ., git commit, dan git push otomatis ke repositori GitHub Anda!